4.6.3 The nested data type

Our previous example showed that a `search` query didn't honor individual document integrity. We can fix this problem by introducing the `nested` data type. The `nested` type is a specialized form of an `object` type that maintains the relationship between arrays of objects in a document.

Continuing our example of emails and attachments, let's define the `attachments` field as the `nested` data type rather than letting Elasticsearch derive it as an `object` type. This calls for creating a schema by declaring the `attachments` field as a `nested` data type.

Listing 4.29 Mapping the schema definition for the `nested` type

```
PUT emails_nested
{
  "mappings": {
    "properties": {
      "attachments": {
        "type": "nested",
        "properties": {
          "filename": {
            "type": "keyword"
          },
          "filetype": {
            "type": "text"
          }
        }
      }
    }
  }
}
```

Declares the attachments field as a nested type

The filename field is declared as a keyword type to avoid tokenizing.

We can leave this field as text.

In addition to creating the `attachments` field as a `nested` type, we declared `filename` as a `keyword` type. This field's value is tokenized: for example, `file1.txt` is split into `file1` and `txt`. As a result, a search query may be matched with `txt` and `confidential` or `txt` and `private`, as both records have `txt` as a common token. To avoid this, we can use `filename` as a `keyword` field. You can also see this method in listing 4.28, where we use `attachments.filename.keyword` in our `search` query.

Let's return to the problem at hand: we have a schema definition, so all we need to do is index a document.

Listing 4.30 Indexing a document with the `nested` property

```
PUT emails_nested/_doc/1
{
  "attachments" : [
    {
      "filename" : "file1.txt",
      "filetype" :  "confidential"
    },
```

Provides a couple of objects as attachments